

CPG: Fundamentals Bootcamp Syllabus

Intro/course description

Dates: Monday, September 21st – Friday, September 25 (Daily)

Time: 07:30p EST – 9p EST

Description: This introductory course provides students with the foundational tools and workflows used in modern software development, cloud engineering, DevOps, and IT operations. Over five sessions, students will progress from configuring a local development environment to working with Linux, GitHub, Bash, Python, and basic automation concepts.

The course emphasizes hands-on practice rather than isolated theory, with each section building on skills introduced during the previous session. Students will learn not only how individual tools work, but also how those tools interact within a typical development workflow. By the end of the course, students will combine Linux commands, scripts, Python code, and file management into a small orchestration workflow similar to those used in real technical environments.

Section I. IDE and File management

This section introduces students to the development environment they will use throughout the course and establishes foundational computer file-management concepts. Students will configure the required software, become familiar with Visual Studio Code, and learn how files and directories are represented and organized on their local machine. Particular attention will be given to absolute and relative file paths, file extensions, directory structures, and navigating a software project from an IDE. These concepts directly reflect workplace expectations, where developers routinely move between project folders, configuration files, source code, terminals, and repositories.

Class lab:

- **Development environment setup:** Install and verify required software, including Visual Studio Code, Git, Python, and the terminal environment. Students will confirm that each tool can be launched and accessed successfully.
- **File and directory navigation:** Create a simple project directory and practice opening, creating, renaming, moving, and deleting files using both the operating system and Visual Studio Code.

- *Hello World HTML exercise:* Create a basic HTML document in VS Code and open it in a browser. Students will use this exercise to connect source files, file extensions, directory paths, and application execution.

HW:

- [Linux KillrCoda \(Lessons 1, 3, 4\)](#)
- The Linux Command Line: Read Chapters 1-3
- [VS Code YT video](#)
- [Filepaths YT videos](#)

Section II. Gitops + Linux CLI

This section introduces the Linux command line alongside the basic Git and GitHub workflow commonly used by software development teams. Students will learn to navigate and manipulate files without relying exclusively on a graphical interface while also learning how source code is tracked through version control. They will practice the relationship between a local Git repository and its remote GitHub counterpart, including branches, commits, pushes, and pull requests. In the workplace, these skills underpin collaborative development, code review, infrastructure management, deployment pipelines, and nearly every modern DevOps workflow.

Class lab:

- *Linux CLI navigation:* Use commands such as pwd, ls, cd, mkdir, touch, cp, mv, cat, and rm to navigate and manipulate a project directory from the terminal.
- *Local and remote Git repositories:* Initialize a local Git repository, create a GitHub repository, configure the remote connection, and push locally committed files to GitHub.
- *Branch and pull request workflow:* Create a feature branch, modify project files, commit and push the branch, and submit a pull request. Students will see how this process supports team collaboration and controlled changes to shared codebases.

HW:

- [Linux KillrCoda \(Lessons 6, 7\)](#)
- The Linux Command Line: Read Chapters 4-5
- [Github YT Video](#)

Section III. Linux scripting

This section moves students from issuing individual Linux commands toward combining those commands into reusable Bash scripts. Students will learn how scripts allow repetitive terminal operations to be documented, automated, and executed consistently. The section also introduces permissions, executable files, shell variables, basic script structure, and terminal-based editing with Vim. Bash scripting remains widely used in systems administration, cloud environments, CI/CD pipelines, server initialization, developer tooling, and operational automation.

Class lab:

- Create a Bash script: Write a small .sh script that declares variables and executes several Linux commands as a repeatable workflow.
- Execute and troubleshoot scripts: Add execute permissions with chmod, run the script from the terminal, and inspect basic errors caused by paths, permissions, or incorrect commands.
- Vim fundamentals: Open and modify a file using Vim while practicing command mode, insert mode, saving, and exiting. Students will discuss why terminal editors remain useful when working with remote Linux servers and cloud systems.

HW:

- Python distilled (Chapters 1-2)
- [Python Killr Coda](#) (Week 1)
- [Python YT Video](#)

Section IV. Python intro

This section introduces Python as the course's first general-purpose programming language. Students will learn how Python represents data using variables, makes decisions with conditional statements, and organizes reusable behavior through functions. The emphasis will be on understanding program execution and learning to read simple code rather than memorizing large amounts of syntax. These same concepts form the foundation of Python applications used for automation, data engineering, cloud administration, APIs, testing, and backend development.

Class lab:

Variables and data: Create variables containing strings, numbers, and Boolean values, then print and modify those values during program execution.

Conditional logic: Use if, elif, and else statements to change program behavior based on variable values and user input.

Functions: Define and call a simple Python function that accepts information, performs an operation, and returns or displays a result. Students will connect functions to the reusable application logic found in production software.

HW:

- Python distilled (Chapters 3-4)
- [Python Killr Coda](#) (Week 5)

Section V. Orchestration scripting (conceptual grounding)

This final section combines the week's Linux, Git, file-management, Bash, and Python concepts into a single coordinated workflow. Students will explore orchestration as the process of using one program or script to control multiple tasks rather than manually executing each operation. The lab will demonstrate how Python can invoke scripts, process information, make decisions, and coordinate actions performed by the operating system. Similar patterns appear in DevOps pipelines, cloud automation, ETL processes, scheduled jobs, infrastructure provisioning, application deployment, and systems administration.

Class lab:

- *Build the orchestration workflow:* Create a Python program that uses variables, functions, conditionals, and simple user input to determine which operation should run.
- *Integrate Python and Linux:* Have the Python program execute an external Bash script or Linux command and observe how separate scripts can work together as components of a larger process.
- *Run the complete workflow:* Test the orchestration process from the terminal, inspect its output, troubleshoot errors, and commit the completed project to Git. Students will finish with a small but complete example of the same layered automation model used in professional engineering environments.

HW:

- Read Cloud without Compromise chapters 1-5 (preparation for Intro to AWS with Blk Muzlim)